

NLP

RNN LSTM y GRU

Docente:
Josselyn Ordóñez

De embeddings a secuencias: ¿qué nos falta?



La clase pasada: cada palabra es un **vector con significado** (embeddings). Pero cuando promediamos vectores o usamos bolsas de palabras, perdemos algo esencial:

“El perro mordió al hombre”

“El hombre mordió al perro”

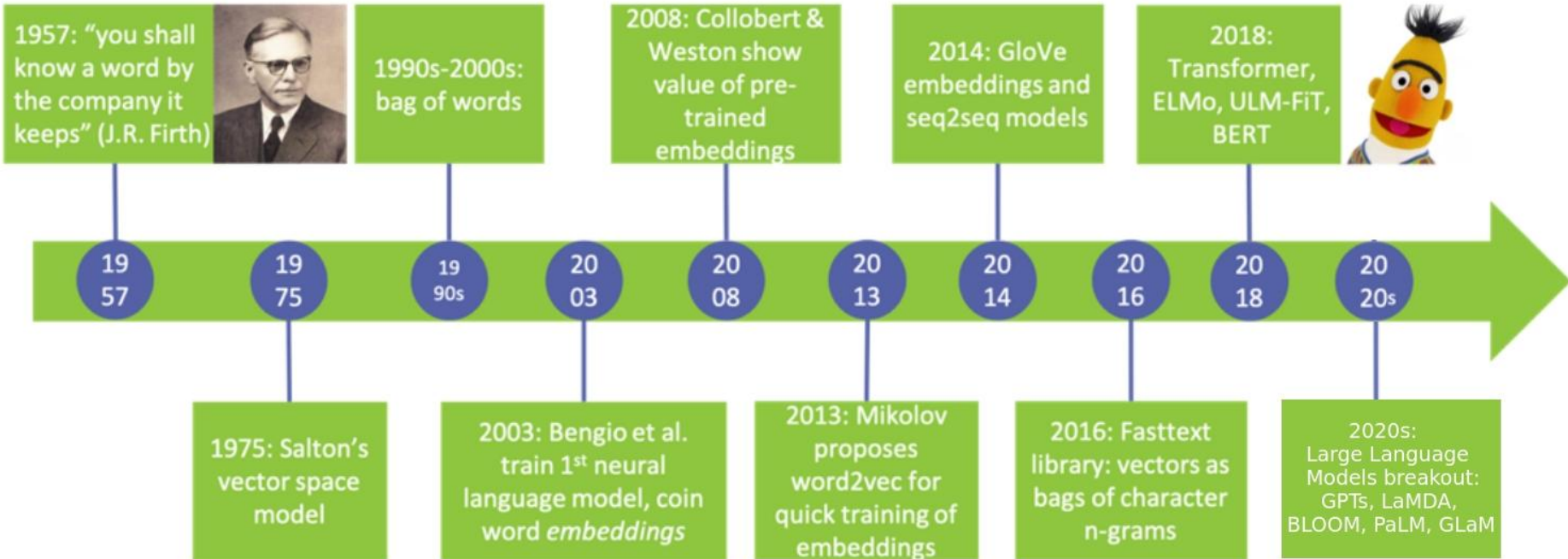
Mismas palabras → misma bolsa de palabras → **mismo embedding promedio**... pero significado opuesto.

El ORDEN importa. Hoy: redes que leen la secuencia palabra por palabra y recuerdan lo que ya vieron → **RNN**

Timeline



1990: Celda RNN básica (Elman)
1997: LSTM



2016: LSTMs se convierten en SotA para traducción automática

Redes Neuronales Recurrentes (RNNs)



Es un tipo de neurona con un estado interno (o memoria) de manera que la información del pasado influye en los resultados futuros.



Se utiliza principalmente para resolver problemas de secuencia, en donde el valor anterior está relacionado con el valor futuro.



Permite construir modelos cuyos tamaños de secuencia sean potencialmente arbitrarios.

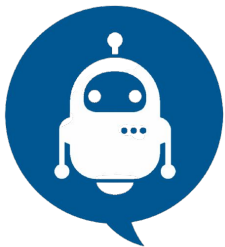


Implementa modelos de lenguaje de la forma:

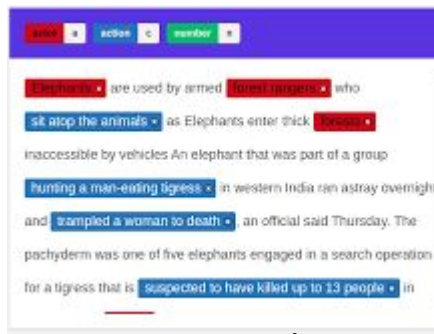
$$\prod_{i=1}^{i=m} P(w_i | w_{i-(n-1)}, \dots, w_{i-1})$$

*"Hoy el **día** está **hermoso** y **despejado**, se puede ver un hermoso **cielo... azul**"*

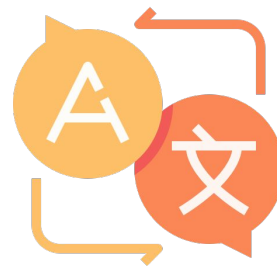
Algunos problemas de secuencia



Bots
Conversacionales



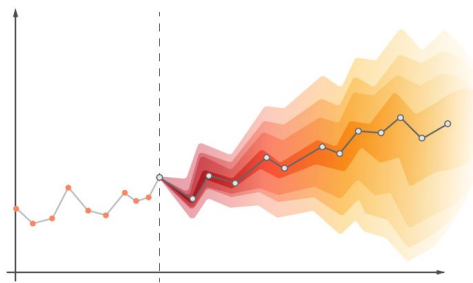
Name entity
recognition



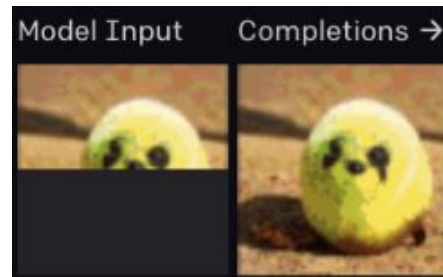
Traducción de
idiomas



Speech to text



Series
temporales



Completar una
imagen

Intuición: la caja del supermercado



La cajera no memoriza todos los precios que ya pasaron. Mantiene **una sola cosa** en la cabeza —el total acumulado— y lo actualiza con cada producto que llega.

Entrada (xt): el precio del producto que llega ahora

Estado (ht): el total acumulado hasta este momento — un resumen de todo lo que ya pasó

Actualización: total nuevo = $f(\text{total anterior, precio nuevo})$ — la misma regla en cada paso

Una RNN hace exactamente esto con palabras: $h_t = \sigma(W_{hh} \cdot h_{t-1} + W_{hx} \cdot x_t + b)$

Celda RNN básica (Elman)

[LINK](#)

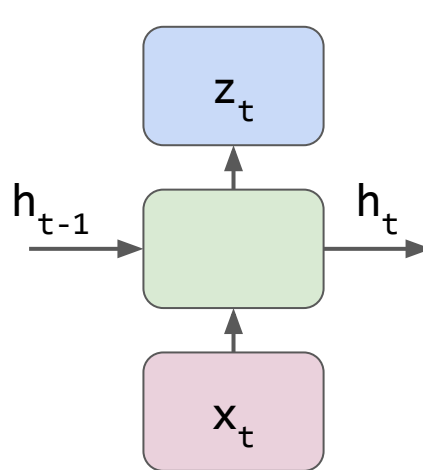
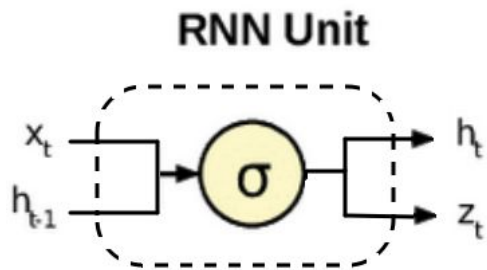
[API KERAS](#)



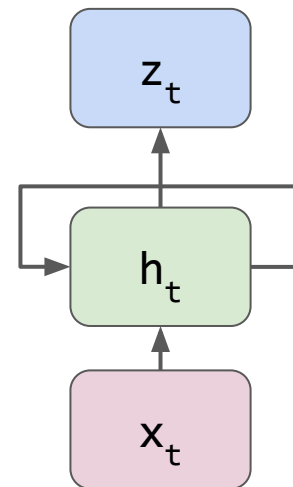
En Keras: SimpleRNN

$$h_t = \sigma(W_{hh} * h_{t-1} + W_{hx} * x + b_h)$$

$$z_t = h_t$$



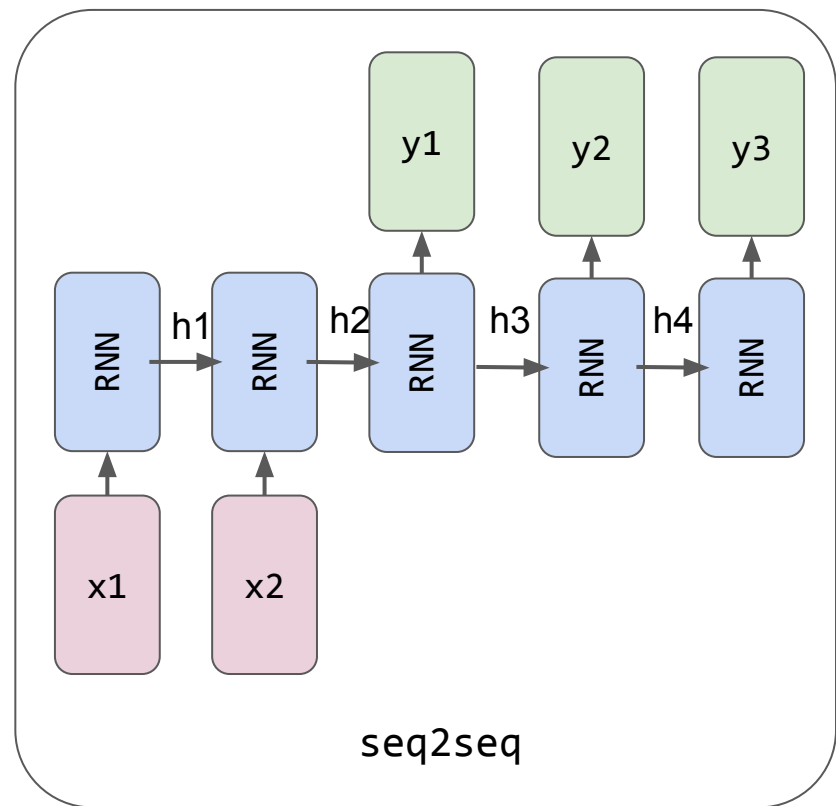
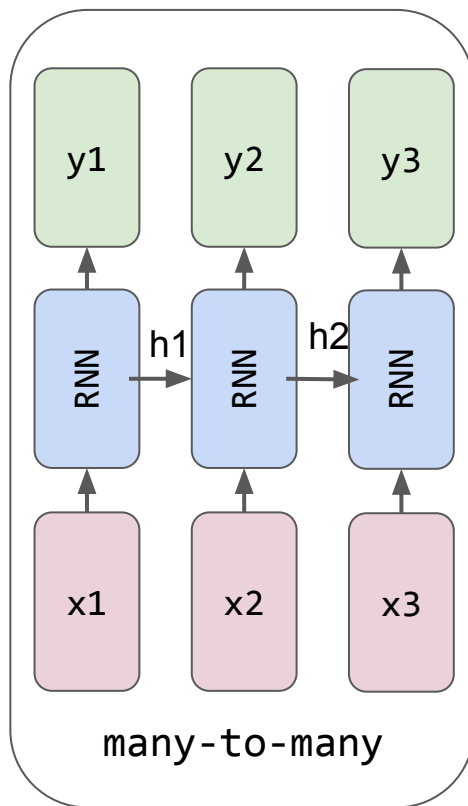
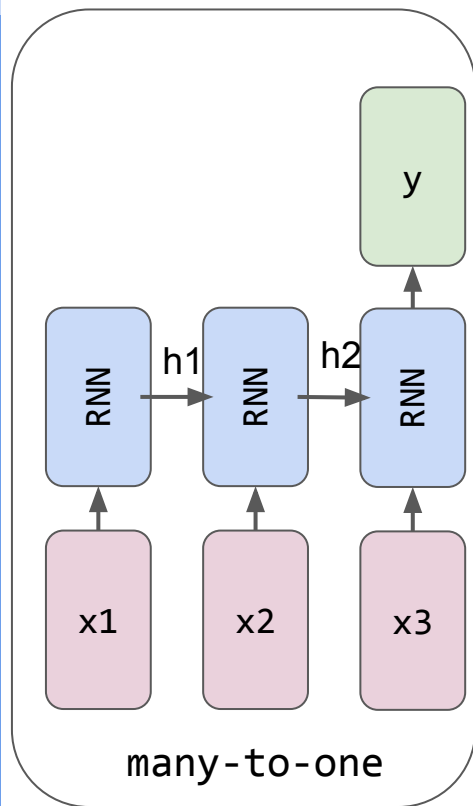
Unidad básica



Representación
compacta

Tipos de problemas de secuencia

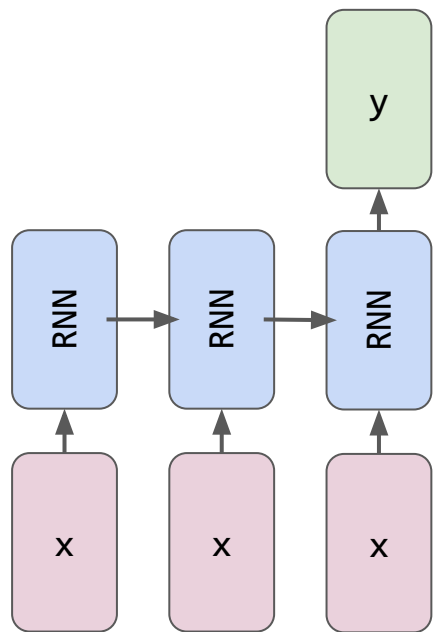
[LINK](#)



many-to-one



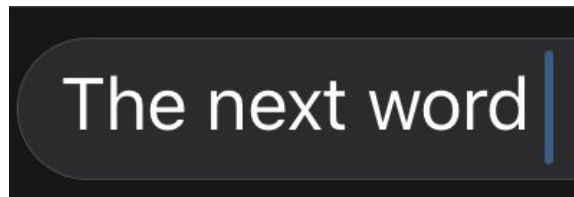
"Dada una sentencia o oración de entrada de tamaño fijo, el sistema arroja un único resultado que la representa".



many-to-one



Este tipo de estructuras se utilizan para determinar cuál es la siguiente palabra o elemento en la secuencia o para clasificación (sentiment analysis).



Predicción de
próxima palabra

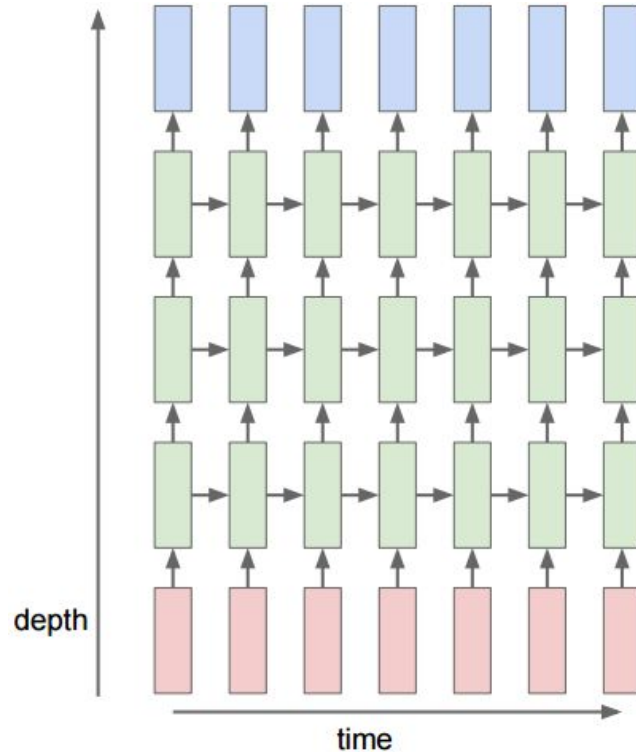


Análisis de
sentimientos

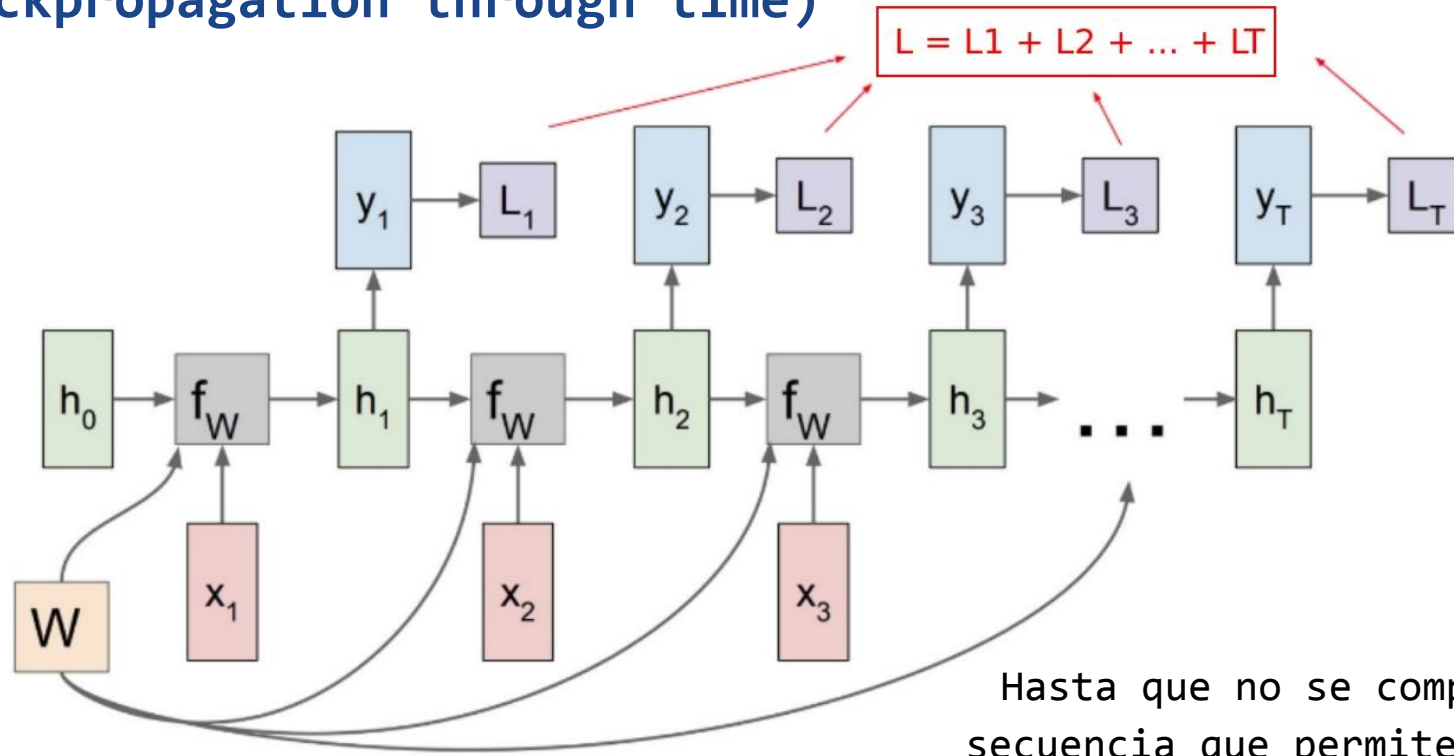
Multi-layer RNN



Tal como se vio en los ejemplos se trata de apilar layers RNN en donde la salida de una se traslada a la entrada de la siguiente



Grafo de cómputo de una RNN y BPTT (Backpropagation through time)



Hasta que no se complete la
secuencia que permite calcular
el loss no se actualizan los
pesos (W) de la/s celda/s



Intervalo

El problema: memoria corta



“Crecí en Francia, pasé toda mi infancia allí, estudié en sus escuelas y jugué en sus plazas... por eso hablo perfectamente _____”

Todos completaron la palabra... porque recuerdan **“Francia”**, a más de 20 palabras de distancia. Una RNN simple no puede: en cada paso el estado se multiplica por W y pasa por σ , y la señal se atenúa un poco más.

Es el **teléfono malogrado**: cada persona repite el mensaje al siguiente, y tras muchos pasos el mensaje original ya se perdió.

Gradientes que se desvanecen (vanishing gradients): la RNN aprende dependencias cortas pero olvida las largas. Necesitamos una celda con memoria de largo plazo → **LSTM**

Long short term memory (LSTM)

[LINK](#)



Se introduce este tipo de celda neuronal con mayor persistencia de memoria para lograr capturar relaciones de palabras a largo plazo.



Se crearon en 1997. Se adoptó como la layer principal para problemas de secuencia en 2014 hasta la aparición de los transformers en 2017.



Desplazaron completamente a las capas RNN simples (Elman), ya que el costo adicional de las LSTM es marginal respecto al beneficio que otorgan



Se basan en el principio de ponderar la importancia de una palabra respecto al contexto futuro/pasado (key words).

¿Comprarías este producto?

*"**Incredible!** El producto es lo que venden, hace lo que tiene que hacer y me **ayudó mucho** a resolver los problemas que tenía. Lo **volvería a comprar** sin dudas"*

Intuición: tomar apuntes en clase



Nadie transcribe la clase completa. Con el cuaderno tomamos **tres decisiones** todo el tiempo:

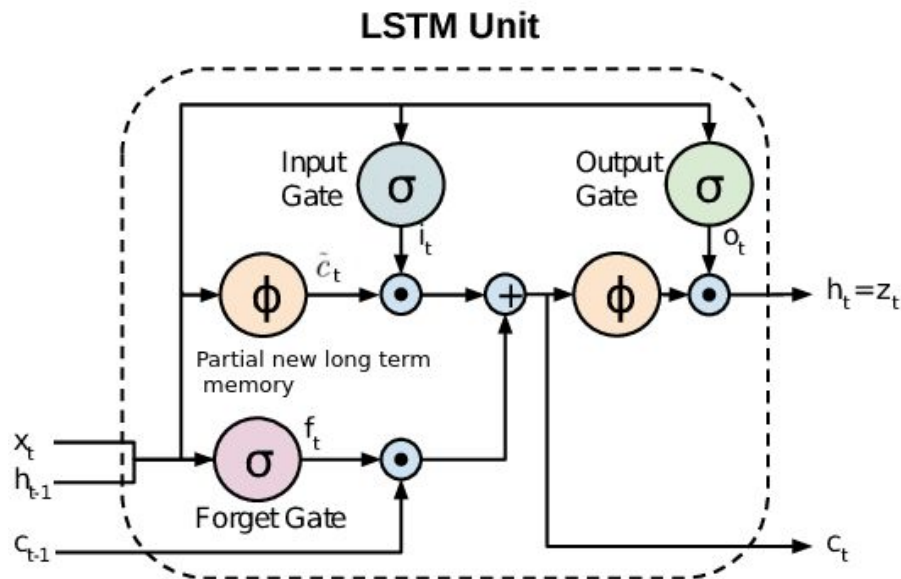
¿Qué anoto de lo nuevo que escucho? → **compuerta de ENTRADA (i)**

¿Qué apunte viejo tacho porque ya no sirve? → **compuerta de OLVIDO (f)**

¿Qué parte del cuaderno leo para responder ahora? → **compuerta de SALIDA (o)**

El cuaderno = memoria de largo plazo (C). Lo que tienes en mente en este instante = estado oculto (h). Las compuertas deciden qué entra, qué se borra y qué se lee.

LSTM approach - Idea de la arquitectura



C: 'Memoria a largo plazo'

Input (i): Dado el último estado (h_{t-1}) evalúa cuánto de la nueva entrada (x) se incluirá en la memoria de largo plazo (C_t).

Forget (f): Dada la entrada (X) cuánto del estado de memoria anterior (C_{t-1}) tiene importancia en el nuevo estado.

Output (o): Qué parte del estado de memoria (C_t) pasa al próximo estado de memoria h_t .

$\cdot$: Producto elemento a elemento

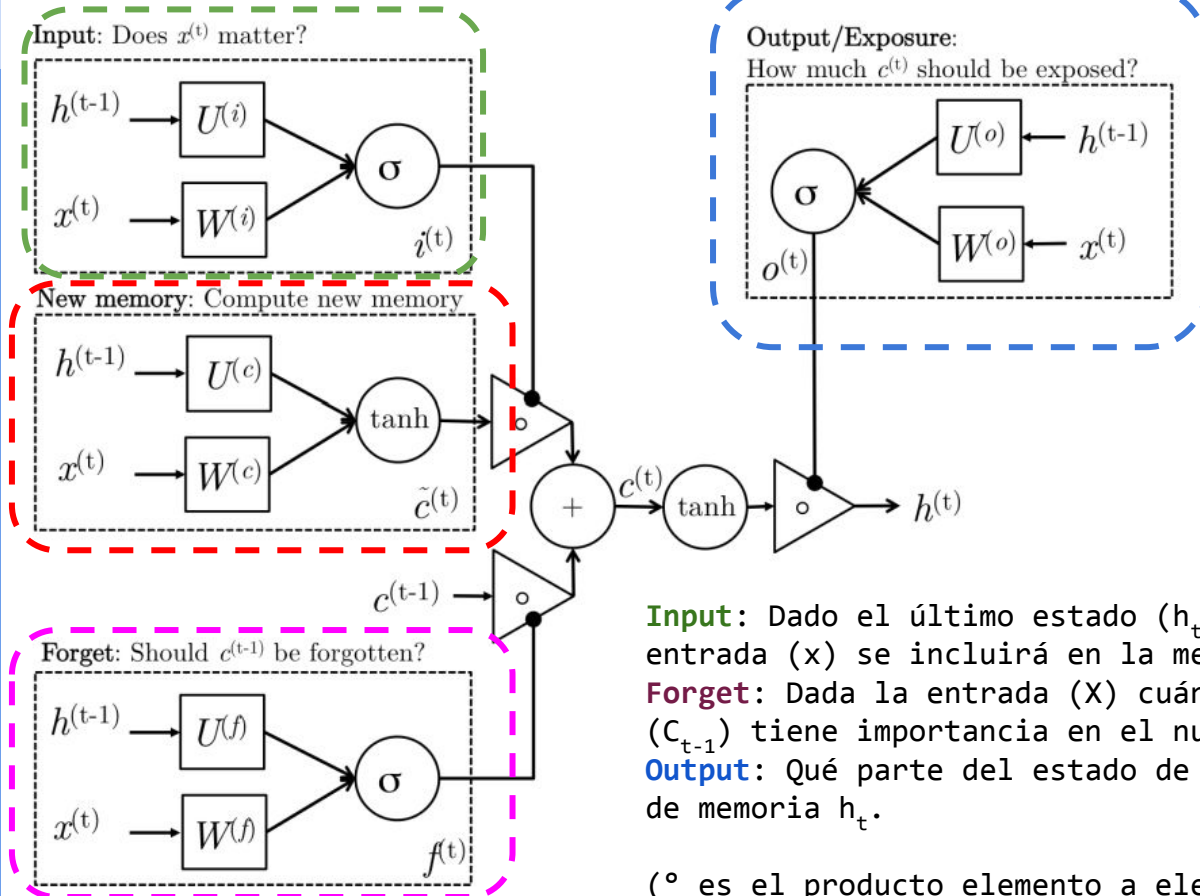
ϕ : Función Tanh

$+$: Suma vectorial

$\rightarrow$: Conexiones

LSTM approach

[LINK](#)



$$i_t = \sigma(W^{(i)}x_t + U^{(i)}h_{t-1})$$

$$f_t = \sigma(W^{(f)}x_t + U^{(f)}h_{t-1})$$

$$o_t = \sigma(W^{(o)}x_t + U^{(o)}h_{t-1})$$

$$\tilde{c}_t = \tanh(W^{(c)}x_t + U^{(c)}h_{t-1})$$

$$c_t = f_t \circ c_{t-1} + i_t \circ \tilde{c}_t$$

$$h_t = o_t \circ \tanh(c_t)$$

Input: Dado el último estado (h_{t-1}) evalúa cuánto de la nueva entrada (x) se incluirá en la memoria de largo plazo (C_t).

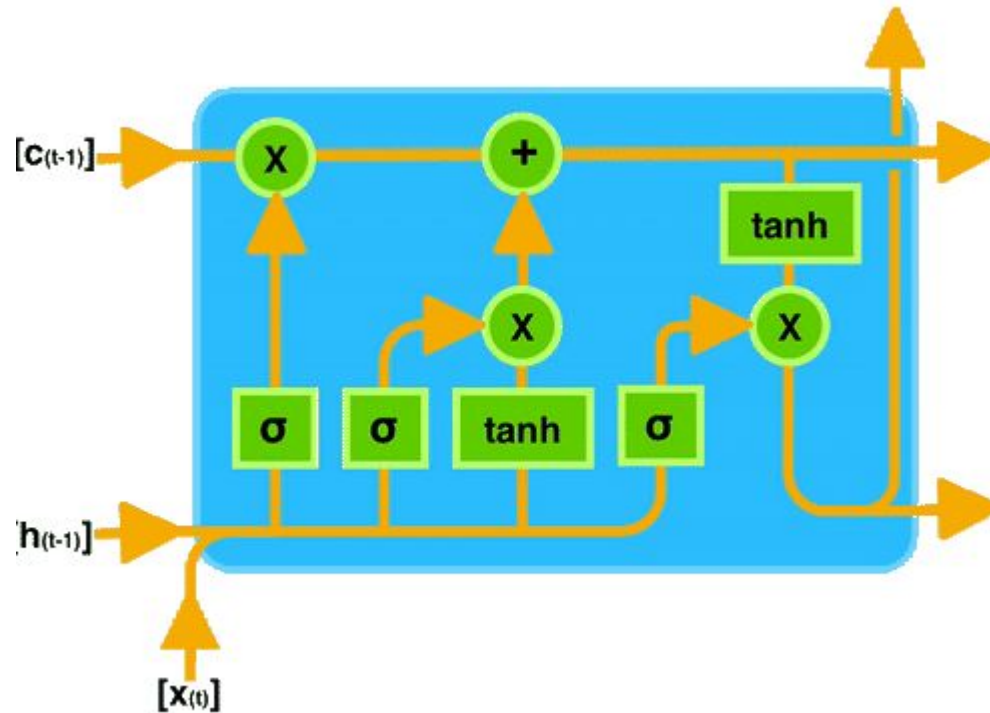
Forget: Dada la entrada (x) cuánto del estado de memoria anterior (C_{t-1}) tiene importancia en el nuevo estado.

Output: Qué parte del estado de memoria (C_t) pasa al próximo estado de memoria h_t .

($\circ$ es el producto elemento a elemento)

Long short term memory (LSTM)

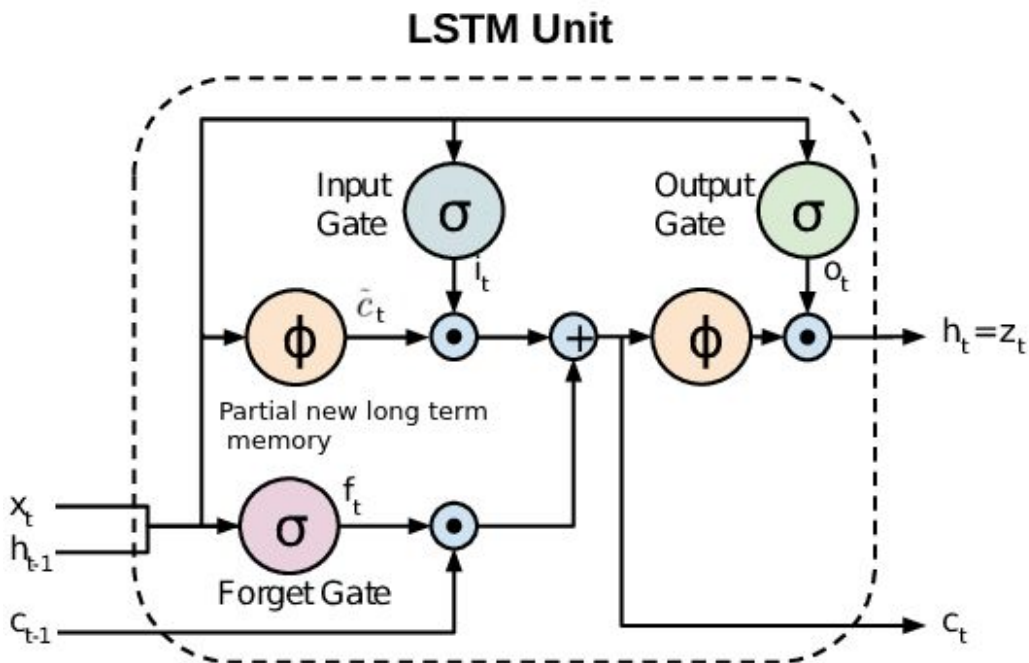
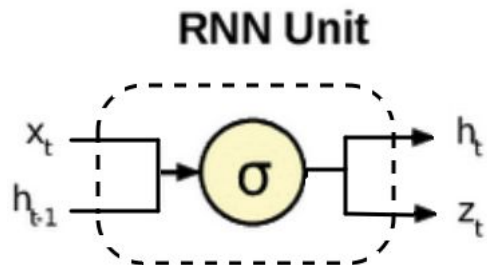
[LINK](#)



LSTM vs RNN



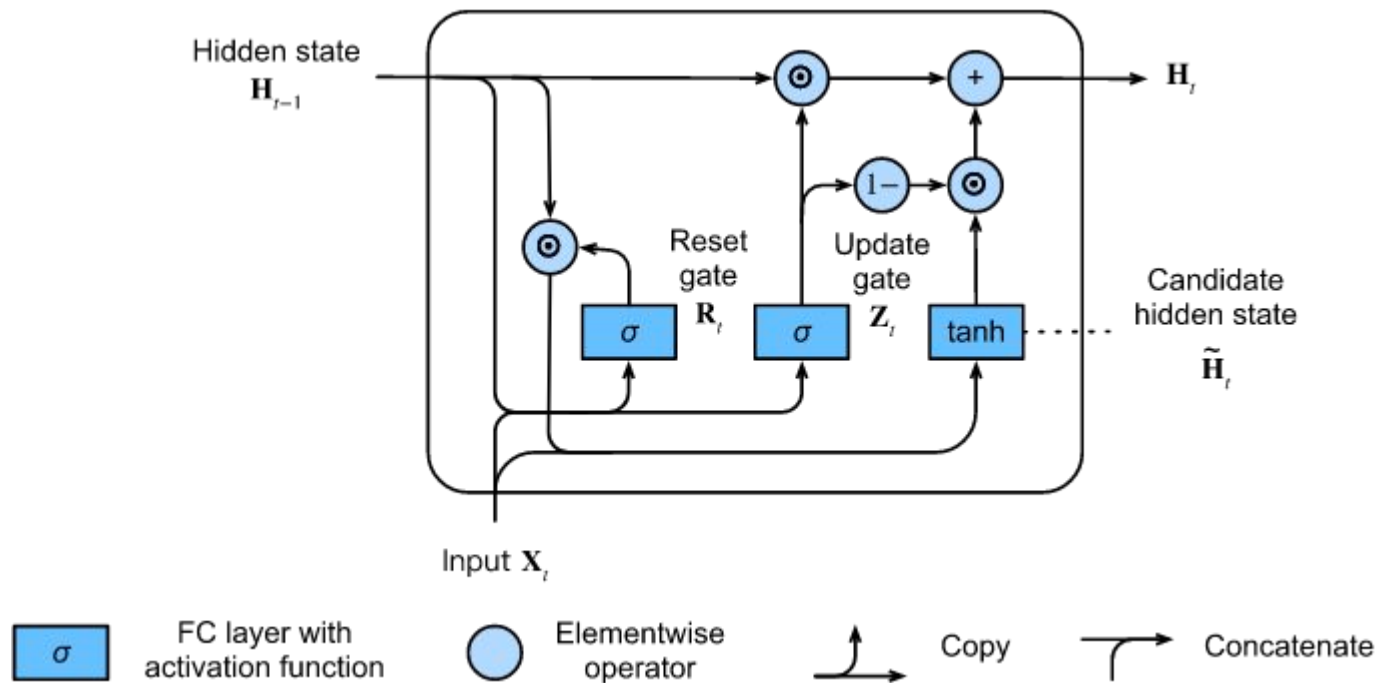
La memoria de largo plazo c_t permite propagar gradiente eficientemente a mayor “profundidad temporal”.



Gated Recurrent Units (GRU)

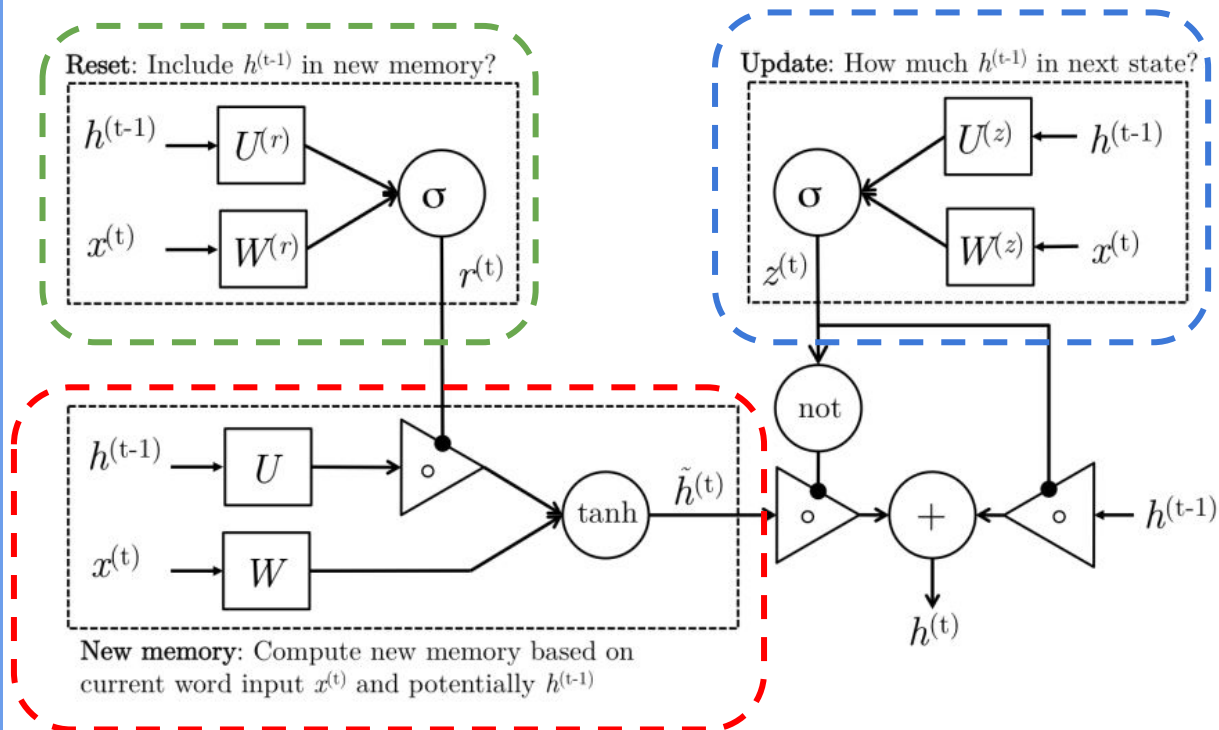


"Evolución de las RNN para superar problemas de "short-memory", versión reducida de una LSTM".



Gated Recurrent Units (GRU)

[LINK](#)



$$z_t = \sigma(W^{(z)}x_t + U^{(z)}h_{t-1})$$

$$r_t = \sigma(W^{(r)}x_t + U^{(r)}h_{t-1})$$

$$\tilde{h}_t = \tanh(r_t \circ U h_{t-1} + W x_t)$$

$$h_t = (1 - z_t) \circ \tilde{h}_t + z_t \circ h_{t-1}$$

(Update gate)

(Reset gate)

(New memory)

(Hidden state)

Problema de una RNN tradicional

[LINK](#)



"Una RNN tradicional solo usa información del pasado y no de las futuras palabras para predecir"

Ejemplo: Data una sentencia determinar si existe una entidad que represente al nombre de una persona utilizando (name entity recognition)

Ejemplo 1:

*"Hoy escuche que **Victoria** terminó su bot para NLP" → persona*

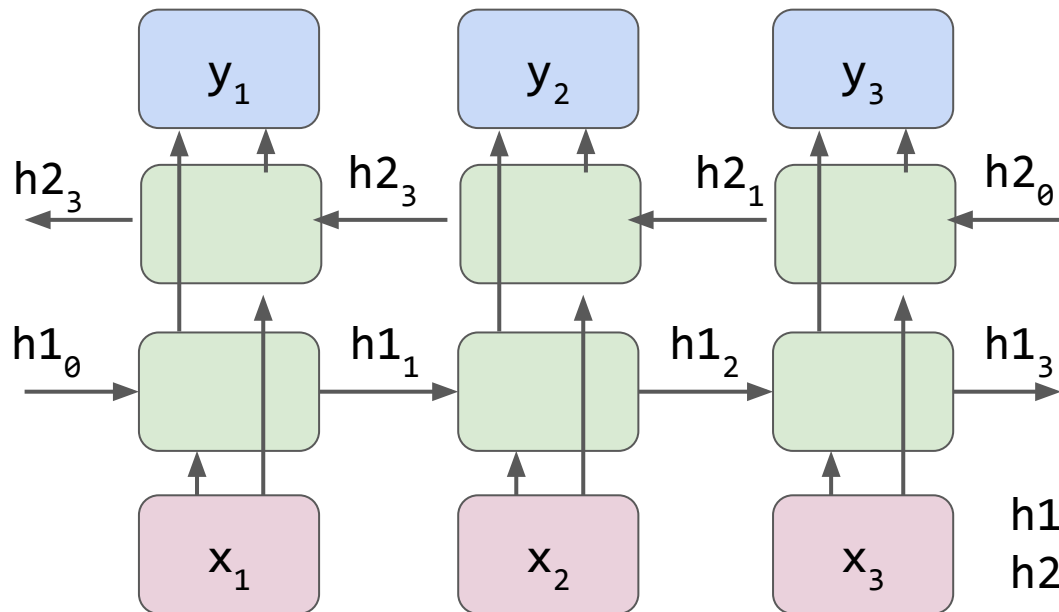
Ejemplo 2:

*"Hoy escuche que **Victoria** cambió de intendente" → ciudad*

La
contextualización
de la palabra es
a futuro



“La palabra anterior y la palabra futura tienen impacto en la presente predicción”



$$h1_t = \sigma(W_{hh1} * h1_{t-1} + W_{hx1} * x + b_1)$$
$$h2_t = \sigma(W_{hh2} * h2_{t+1} + W_{hx2} * x + b_2)$$

Las dos salidas pueden concatenarse, sumarse o promediarse



Intervalo

Predicción de texto - Modelos de lenguaje



Objetivo: Entender la estructura estadística del lenguaje aprovechando su estructura secuencial.

Tarea: Aprender a predecir la siguiente palabra.

$$\prod_{i=1}^{i=m} P(w_i | w_{i-(n-1)}, \dots, w_{i-1})$$

The next word |

$$(X_1, \dots, X_n) \longrightarrow X_{n+1}$$

Pasos a seguir:

